

01 — Normalization: 1NF, 2NF, and 3NF

Table of Contents

1. [Learning Objectives](#)
 2. [Why Normalize?](#)
 3. [Functional Dependencies](#)
 4. [First Normal Form \(1NF\)](#)
 5. [Second Normal Form \(2NF\)](#)
 6. [Third Normal Form \(3NF\)](#)
 7. [Progressive Normalization Example](#)
 8. [Anomalies Reference](#)
 9. [SQL Examples](#)
 10. [Common Mistakes](#)
 11. [Best Practices](#)
 12. [Performance Considerations](#)
 13. [Interview Questions & Answers](#)
 14. [Exercises with Solutions](#)
 15. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Identify functional dependencies in a relation
 - Detect 1NF, 2NF, and 3NF violations and explain exactly what the rule is
 - Transform any unnormalized table through 1NF → 2NF → 3NF
 - Explain the specific anomalies each normal form eliminates
 - Write the correct normalized PostgreSQL DDL for any given problem
-

Why Normalize?

Un-normalized tables suffer from three types of anomalies:

UPDATE anomaly:	Changing one real-world fact requires updating multiple rows. If any row is missed, data becomes inconsistent.
INSERT anomaly:	Cannot add certain facts without adding unrelated facts first. (e.g., cannot add a department until it has at least one employee)
DELETE anomaly:	Deleting a row removes more information than intended. (e.g., deleting the last employee in a dept also deletes dept info)

Normalization eliminates these anomalies by ensuring:

1. Each fact is stored **exactly once**
 2. Each attribute depends **only on what it should depend on**
-

Functional Dependencies

A **functional dependency** $X \rightarrow Y$ means: knowing X uniquely determines Y.

Examples:		
student_id	→	student_name (knowing ID determines the name)
student_id	→	date_of_birth (knowing ID determines DOB)
course_id	→	course_name (knowing course ID determines name)
(student_id, course_id)	→	grade (composite: grade depends on both)
NOT a functional dependency:		
student_name	→	student_id (two students can have same name)
course_name	→	instructor (same course may have multiple instructors)

Types of functional dependencies:

- **Full FD:** Y depends on the whole of X (no subset of X determines Y)

- **Partial FD:** Y depends on part of X when X is composite
- **Transitive FD:** $X \rightarrow Y \rightarrow Z$ (Z depends on X indirectly, through Y)

First Normal Form (1NF)

Rule

A table is in 1NF if:

1. Each column contains **atomic (indivisible) values**
2. Each column contains values of a **single type**
3. Each row is **unique** (there is a primary key)
4. There are **no repeating groups** (columns like phone1, phone2, phone3)

Violation examples

Violation 1: Multi-valued cells

UNNORMALIZED:

order_id	customer	items	
1001	Alice	"Pen x2, Notebook x1"	← not atomic!
1002	Bob	"Stapler x3"	

Violation 2: Repeating groups (columns)

UNNORMALIZED:

student_id	name	phone1	phone2	phone3	
1	Alice	555-01	555-02	NULL	← repeating groups!
2	Bob	555-03	NULL	NULL	

1NF Fix

1NF – Remove repeating groups by creating separate rows:

student_id	name	phone	
1	Alice	555-01	← Alice appears twice
1	Alice	555-02	
2	Bob	555-03	

PK: (student_id, phone)

```
-- BAD: 1NF violation
CREATE TABLE students_bad (
  student_id INTEGER PRIMARY KEY,
  name       TEXT,
  phone1     TEXT, -- repeating group
  phone2     TEXT,
  phone3     TEXT
);

-- GOOD: 1NF compliant
CREATE TABLE students (
  student_id INTEGER NOT NULL,
  name       TEXT      NOT NULL,
  phone      TEXT      NOT NULL,
  PRIMARY KEY (student_id, phone)
);

-- OR better: separate phones into their own table (leading toward 2NF)
CREATE TABLE student_phones (
```

```

student_id INTEGER NOT NULL,
phone      TEXT      NOT NULL,
phone_type TEXT      NOT NULL DEFAULT 'mobile',
PRIMARY KEY (student_id, phone)
);

```

Second Normal Form (2NF)

Rule

A table is in 2NF if:

1. It is in 1NF
2. **Every non-key attribute is fully functionally dependent on the entire primary key** (no partial dependencies)

2NF only applies to tables with **composite primary keys**. A table with a single-column primary key is automatically in 2NF if it is in 1NF.

Violation example

ORDER_ITEMS table (violates 2NF):

order_id	product_id	product_name	product_price	quantity
1001	P01	Pen	1.50	10
1001	P02	Notebook	5.00	2
1002	P01	Pen	1.50	5

← 'Pen' duplicated

PK: (order_id, product_id)

Functional dependencies:

```

(order_id, product_id) → quantity      ← FULL dependency ✓
product_id              → product_name  ← PARTIAL dependency ✗
product_id              → product_price ← PARTIAL dependency ✗

```

Problems:

```

UPDATE: Pen's price changes → must update every order_items row containing P01
INSERT: Cannot record a product until it appears in an order
DELETE: Deleting last order with P01 removes P01's name and price

```

2NF Fix — decompose on the partial dependency

SPLIT into two tables:

PRODUCTS:

product_id	product_name	price
P01	Pen	1.50
P02	Notebook	5.00

PK: product_id

ORDER_ITEMS:

order_id	product_id	quantity
1001	P01	10
1001	P02	2
1002	P01	5

PK: (order_id, product_id)

FK: product_id → products

```

-- 2NF compliant schema
CREATE TABLE products (
  product_id  CHAR(3)      PRIMARY KEY,
  product_name VARCHAR(100) NOT NULL,
  base_price  NUMERIC(10,4) NOT NULL CHECK (base_price >= 0)
);

CREATE TABLE orders (
  order_id    INTEGER PRIMARY KEY,
  customer_id INTEGER NOT NULL
);

```

```
CREATE TABLE order_items (
  order_id    INTEGER      NOT NULL REFERENCES orders(order_id),
  product_id  CHAR(3)      NOT NULL REFERENCES products(product_id),
  quantity    INTEGER      NOT NULL CHECK (quantity > 0),
  unit_price  NUMERIC(10,4) NOT NULL, -- snapshot of price at time of order
  PRIMARY KEY (order_id, product_id)
);
```

Third Normal Form (3NF)

Rule

A table is in 3NF if:

1. It is in 2NF
2. **Every non-key attribute is non-transitively dependent on the primary key** (no transitive dependencies)

A transitive dependency is: $PK \rightarrow A \rightarrow B$ (B depends on A, A is not a key).

Violation example

EMPLOYEES table (violates 3NF):

emp_id	emp_name	dept_id	dept_name	dept_location
1	Alice	D01	Engineering	Building A
2	Bob	D01	Engineering	Building A
3	Carol	D02	Marketing	Building B

-- duplicated

PK: emp_id

Transitive dependencies:

```
emp_id  → dept_id      ← direct ✓
dept_id → dept_name    ← transitive! (not via emp_id) ✗
dept_id → dept_location ← transitive! ✗
```

So: $emp_id \rightarrow dept_id \rightarrow dept_name$ (transitive chain)

Problems:

```
UPDATE: Rename "Engineering" → update every employee row
INSERT: Cannot add a department without an employee
DELETE: Delete all Engineering employees → lose Engineering dept info
```

3NF Fix — decompose on the transitive dependency

SPLIT the transitive dependency out:

DEPARTMENTS:

dept_id	dept_name	loc
D01	Engineering	Bldg A
D02	Marketing	Bldg B

PK: dept_id

EMPLOYEES:

emp_id	emp_name	dept_id
1	Alice	D01
2	Bob	D01
3	Carol	D02

PK: emp_id

FK: dept_id → departments

```
-- 3NF compliant schema
CREATE TABLE departments (
  dept_id    CHAR(3)      PRIMARY KEY,
  dept_name  VARCHAR(100) NOT NULL UNIQUE,
  location   TEXT
);
```

```
CREATE TABLE employees (  
  emp_id      INTEGER      GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  emp_name    TEXT         NOT NULL,  
  dept_id     CHAR(3)      NOT NULL REFERENCES departments(dept_id),  
  hire_date   DATE         NOT NULL,  
  salary      NUMERIC(10,2) NOT NULL CHECK (salary > 0)  
);
```

Progressive Normalization Example

Let's take a completely unnormalized order tracking table and normalize it step by step.

Starting point: raw data (0NF)

RAW ORDER FORM DATA:

order_id	order_date	cust_id	cust_name	cust_email	cust_city	
items: "P01:Pen:1.50:10, P02:Notebook:5.00:2" total: 25.00						

Step 1: Apply 1NF (atomic values, no repeating groups)

ORDERS_1NF:

order_id	order_date	cust_id	cust_name	cust_email	cust_city	p_id	p_name	price	qty
1001	2024-01-10	C01	Alice	alice@example.com	New York	P01	Pen	1.50	10
1001	2024-01-10	C01	Alice	alice@example.com	New York	P02	Notebook	5.00	2
1002	2024-01-11	C02	Bob	bob@example.com	Chicago	P01	Pen	1.50	5

PK: (order_id, p_id)
Problems remaining: partial and transitive dependencies everywhere

Step 2: Apply 2NF (remove partial dependencies)

Partial dependencies found:
p_id → p_name, price (depends only on p_id, not full PK)
order_id → order_date, cust_id, cust_name, cust_email, cust_city

PRODUCTS:

ORDERS (partial):

p_id	p_name	price	order_id	order_date	cust_id	cust_name	cust_email	cust_city
P01	Pen	1.50	1001	2024-01-10	C01	Alice	alice@...	New York
P02	Notebook	5.00	1002	2024-01-11	C02	Bob	bob@...	Chicago

ORDER_ITEMS:

order_id	p_id	qty
1001	P01	10
1001	P02	2

1002	P01	5
------	-----	---

PK: (order_id, p_id)

Problem: transitive deps in ORDERS (cust_id → name, email, city)

Step 3: Apply 3NF (remove transitive dependencies)

Transitive dependency in ORDERS:

order_id → cust_id → cust_name, cust_email, cust_city

Split customers out:

CUSTOMERS:

ORDERS (3NF):

cust_id	cust_name	cust_email	cust_city
C01	Alice	alice@example.com	New York
C02	Bob	bob@example.com	Chicago

order_id	order_date	cust_id
1001	2024-01-10	C01
1002	2024-01-11	C02

RESULT: 4 tables in 3NF

customers, orders, products, order_items

Each fact stored exactly once!

Anomalies Reference

Normal Form	Eliminates	FD Rule
1NF	Repeating groups, non-atomic values	All values must be atomic Row must be uniquely identifiable
2NF	Partial dependencies (UPDATE/INSERT/DELETE anomalies on partial)	Non-key attrs depend on FULL PK (only relevant with composite PK)
3NF	Transitive dependencies (same anomalies via transitivity)	Non-key attrs depend ONLY on PK (not on other non-key attrs)

SQL Examples

Full 3NF schema for an order management system

```
-- CUSTOMERS (3NF)
CREATE TABLE customers (
  customer_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  first_name TEXT NOT NULL,
  last_name TEXT NOT NULL,
  email TEXT NOT NULL UNIQUE,
  city TEXT,
  country CHAR(2) NOT NULL DEFAULT 'US',
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- CATEGORIES (3NF)
CREATE TABLE categories (
  category_id SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name TEXT NOT NULL UNIQUE,
  parent_id SMALLINT REFERENCES categories(category_id)
);

-- PRODUCTS (3NF)
CREATE TABLE products (
```

```

    product_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sku        TEXT      NOT NULL UNIQUE,
    name       TEXT      NOT NULL,
    category_id SMALLINT NOT NULL REFERENCES categories(category_id),
    unit_price NUMERIC(10,4) NOT NULL CHECK (unit_price >= 0)
);

-- ORDERS (3NF)
CREATE TABLE orders (
    order_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    customer_id BIGINT NOT NULL REFERENCES customers(customer_id),
    status     TEXT     NOT NULL DEFAULT 'pending'
                CHECK (status IN ('pending', 'paid', 'shipped', 'delivered', 'cancelled')),
    ordered_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ORDER_ITEMS (3NF) — junction table, no transitive or partial deps
CREATE TABLE order_items (
    order_id BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id INTEGER NOT NULL REFERENCES products(product_id),
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price NUMERIC(10,4) NOT NULL, -- price snapshot at order time (intentional!)
    PRIMARY KEY (order_id, product_id)
);

-- Indexes
CREATE INDEX idx_orders_customer ON orders (customer_id);
CREATE INDEX idx_order_items_product ON order_items (product_id);

```

Common Mistakes

1. **Confusing 1NF with having "no duplicates"** — 1NF is about atomic values and no repeating groups. Duplicate ROWS violate 1NF (no unique key). Duplicate VALUES in a column (multiple rows with same name) are fine and expected.
2. **Thinking 2NF applies to single-column PKs** — Partial dependency requires a composite key. A table with a single-column PK is always 2NF (if 1NF).
3. **Normalizing computed values out when they shouldn't be** — `unit_price` in `order_items` is intentionally denormalized — it's a snapshot of the price at order time, not a transitive dependency. The price can change; the historical order price should not.
4. **Over-normalizing to the point of impracticality** — Normalizing city to a cities table, cities to states table, etc. creates joins that add complexity without benefit for most applications.
5. **Ignoring functional dependencies — normalizing "by feel"** — Always identify and document functional dependencies before decomposing.

Best Practices

1. **Start by listing all functional dependencies** before writing any DDL.
2. **Normalize to 3NF as the standard target** — it eliminates the most common anomalies.
3. **Keep price snapshots** in transaction tables (`order_items.unit_price`) — this is intentional, not a violation.
4. **Name junction tables descriptively** — `order_items` not `order_product`.
5. **Add FK indexes** on the non-leading columns of every junction table.
6. **Document any intentional denormalization** with comments.

Performance Considerations

Normalization increases the number of tables → more JOINS needed.

```

-- Normalized query (3 JOINS for a common report)
SELECT
    o.order_id,
    c.first_name || ' ' || c.last_name AS customer_name,
    p.name AS product_name,

```

```

    oi.quantity,
    oi.unit_price,
    oi.quantity * oi.unit_price AS line_total
FROM orders o
JOIN customers c  ON c.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p   ON p.product_id = oi.product_id
WHERE o.ordered_at >= '2024-01-01';

-- Proper indexes make this fast:
CREATE INDEX idx_orders_customer  ON orders (customer_id);
CREATE INDEX idx_orders_date      ON orders (ordered_at);
CREATE INDEX idx_order_items_order ON order_items (order_id);
CREATE INDEX idx_order_items_prod ON order_items (product_id);

-- Query plan with indexes: hash join / index nested loop, not seq scans

```

Interview Questions & Answers

Q1: What is a functional dependency?

A: A functional dependency $X \rightarrow Y$ means that knowing the value of X uniquely determines the value of Y . If two rows have the same X , they must have the same Y . Example: `employee_id → employee_name` — knowing an employee's ID tells you their name uniquely.

Q2: What is 1NF and what anomalies does it eliminate?

A: 1NF requires all values to be atomic (indivisible) and each row to be uniquely identifiable. It eliminates: multi-valued cells (comma-separated lists in a column), repeating group columns (phone1, phone2, phone3), and ensures a proper primary key exists.

Q3: Why does 2NF only matter for composite primary keys?

A: Partial dependency means "a non-key attribute depends on only part of the composite primary key." If the PK has only one column, there is no "part" of it — the only dependency is the full key. So a table with a single-column PK is automatically 2NF if it satisfies 1NF.

Q4: What is a transitive dependency and how does 3NF address it?

A: A transitive dependency is $X \rightarrow Y \rightarrow Z$, where Z depends on Y (a non-key attribute) instead of directly on X (the primary key). 3NF says: every non-key attribute must depend directly on the primary key, not on another non-key attribute. Fix: extract Y and Z into their own table with Y as the PK.

Q5: What update anomaly would occur in an unnormalized employees/departments table?

A: If department name or location is stored in each employee row, renaming a department requires updating every row where employees belong to that department. If any update is missed, different rows will show different department names — inconsistent data.

Q6: Is having the same value in multiple rows a normalization violation?

A: No. Multiple employees can have the same city — that's not a problem. Normalization is about functional dependencies, not about duplicate values. The problem is when a single real-world fact (like a department's location) is stored in multiple places and can get out of sync.

Q7: Should order_items.unit_price be normalized to products.price?

A: No! The price in `order_items` is a snapshot of the price at the time of the order. Even though it's "derived" from `products.price`, it represents a different fact: "what was the price when this customer bought it." Product prices change; historical order prices should not change. This is intentional denormalization for audit/historical purposes.

Q8: What is the difference between 2NF and 3NF violations?

A: A 2NF violation is a partial dependency: a non-key attribute depends on only part of a composite PK. A 3NF violation is a transitive dependency: a non-key attribute depends on another non-key attribute (indirectly on the PK). 2NF cannot be violated in a table with a single-column PK; 3NF can be.

Exercises with Solutions

Exercise 1

The following table stores university enrollment data. Identify all functional dependencies, describe which normal form(s) it violates, and normalize it to 3NF.

ENROLLMENT:

student_id | student_name | major_id | major_name | course_id | course_name | grade | instructor_id | instructor_name

Solution:

Functional dependencies:

```
student_id → student_name, major_id
major_id → major_name
course_id → course_name, instructor_id
instructor_id → instructor_name
(student_id, course_id) → grade
```

PK: (student_id, course_id)

Violations:

- **2NF**: student_id → student_name (partial dep on PK)
- **2NF**: course_id → course_name (partial dep on PK)
- **3NF**: student_id → major_id → major_name (transitive)
- **3NF**: course_id → instructor_id → instructor_name (transitive)

3NF schema:

```
CREATE TABLE majors (
    major_id    SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    major_name  TEXT NOT NULL UNIQUE
);

CREATE TABLE students (
    student_id  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_name TEXT NOT NULL,
    major_id    SMALLINT NOT NULL REFERENCES majors(major_id)
);

CREATE TABLE instructors (
    instructor_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    instructor_name TEXT NOT NULL
);

CREATE TABLE courses (
    course_id    INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_name  TEXT NOT NULL,
    instructor_id INTEGER NOT NULL REFERENCES instructors(instructor_id)
);

CREATE TABLE enrollments (
    student_id  INTEGER NOT NULL REFERENCES students(student_id),
    course_id   INTEGER NOT NULL REFERENCES courses(course_id),
    grade       CHAR(2) CHECK (grade IN ('A+', 'A', 'A-', 'B+', 'B', 'B-', 'C+', 'C', 'F')),
    enrolled_at DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (student_id, course_id)
);
```

Exercise 2

Explain why the following is NOT a 3NF violation:

```
CREATE TABLE employees (
    emp_id      INTEGER PRIMARY KEY,
    full_name   TEXT,
    birth_year  INTEGER,
    age         INTEGER -- derived from birth_year!
);
```

Answer: This is not a 3NF violation but rather poor design. 3NF concerns functional dependencies between non-key attributes. `age` is derived from `birth_year + current_year`, but the real issue is that `age` changes every year and should never be stored — it should be computed. A computed column (`GENERATED ALWAYS AS`) would be appropriate if storage is needed. This is a design problem, not strictly a normalization violation.

Cross-References

- `02_bcnf_4nf_5nf.md` — higher normal forms
- `03_denormalization.md` — when to intentionally denormalize
- `04_entity_relationships.md` — FK design that emerges from normalization
- `../05_PostgreSQL_Core/08_constraints.md` — implementing FK constraints